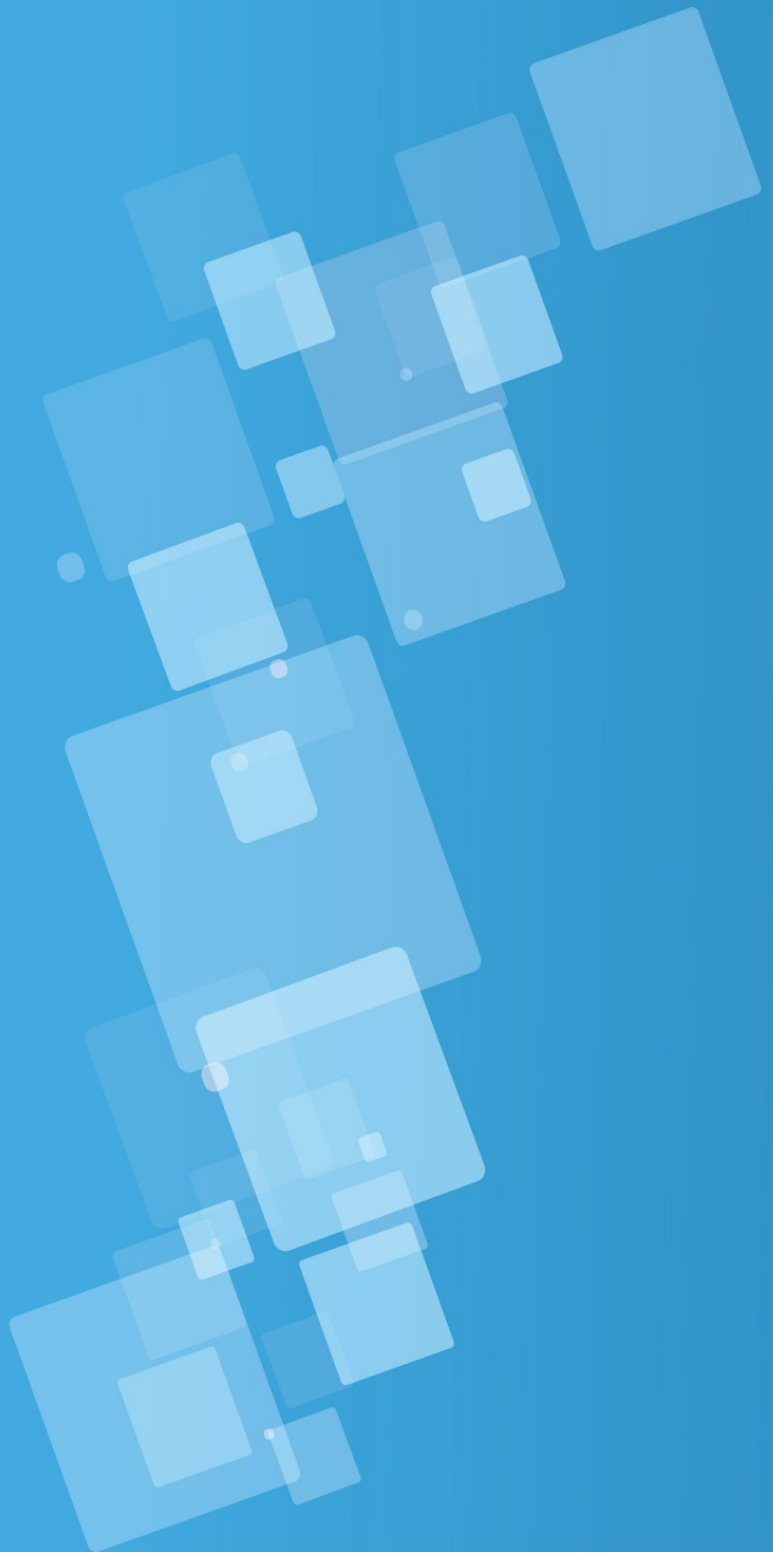
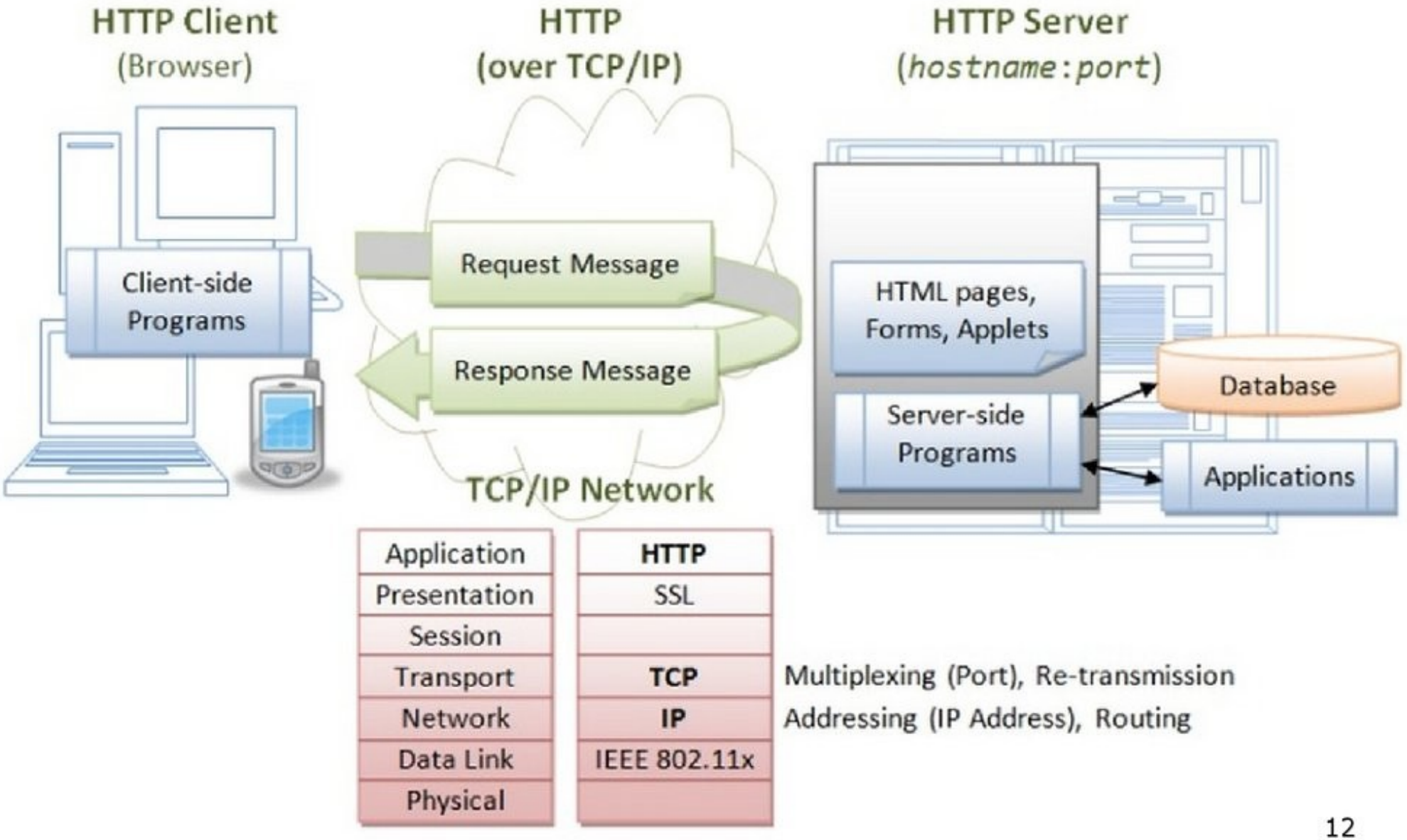


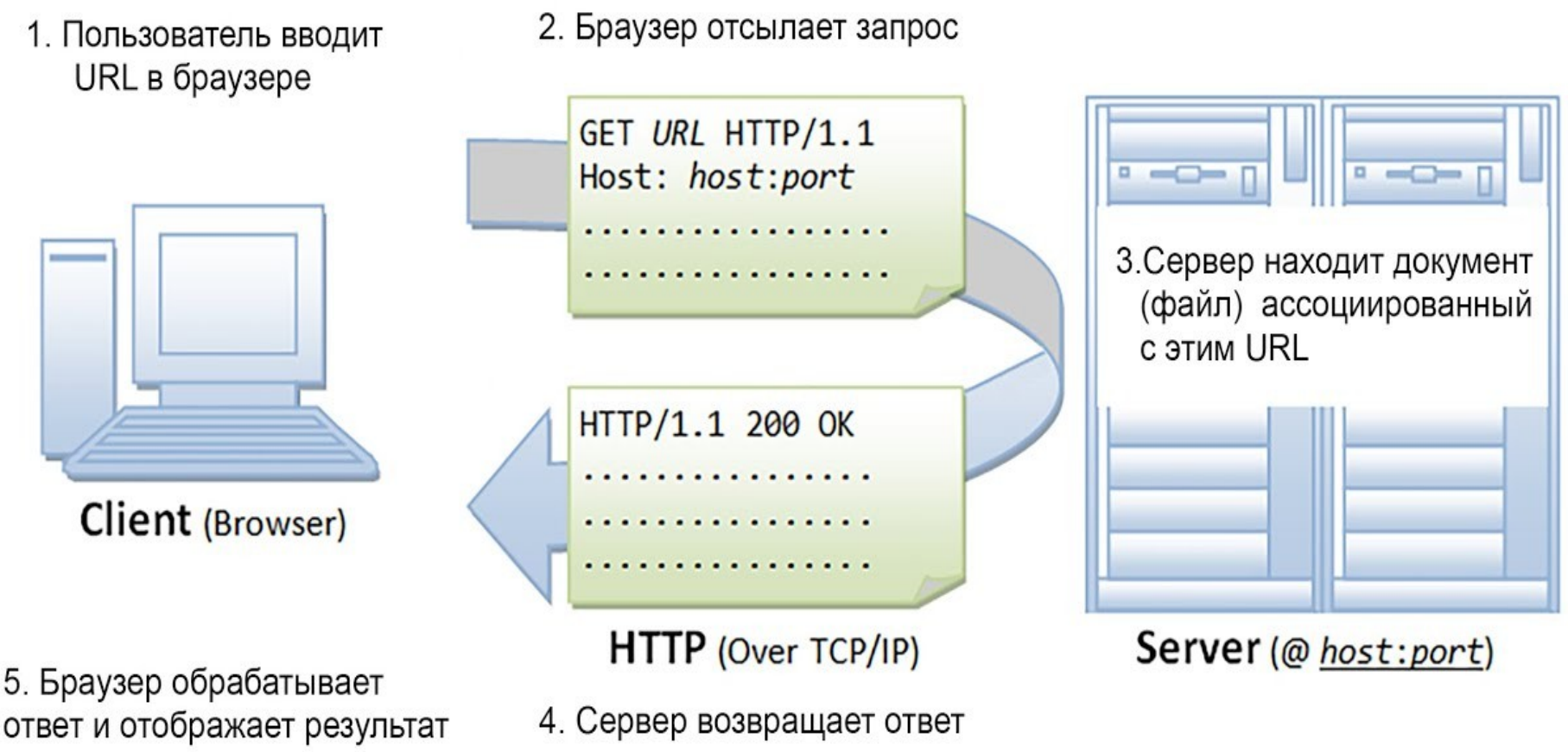
Micro framework Flask

Web services REST & RESTful

Обобщенная схема взаимодействия по протоколу HTTP:



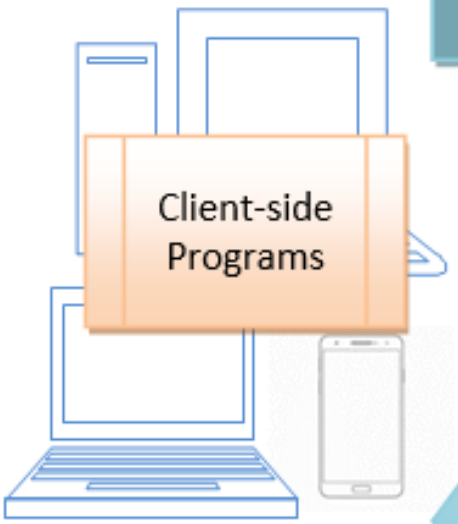
Взаимодействие по HTTP



HTTP over TCP/IP

HTTP Client(s)

http://xyz.com/home.html



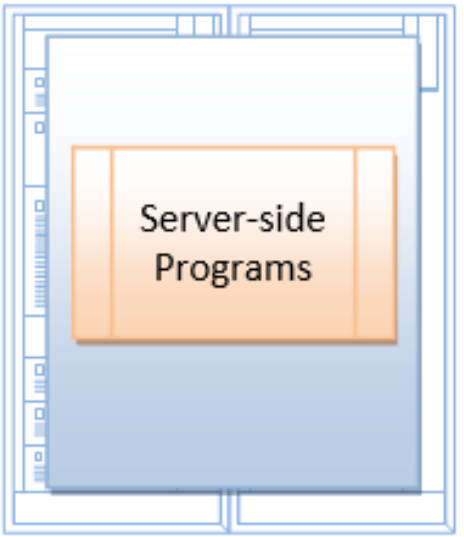
Request Message

```
GET /home.html HTTP/1.1
Host: xyz.com
Connection: Keep-Alive
User-Agent: Mozilla/4.0
Accept: image/gif, image/jpeg
----- Blank line ------
(Empty body)
```

Response Message

```
HTTP/1.1 200 OK
Date: ...
Server: Apache/2.0.45
Last-Modified: ...
Content-Length: 105
Content-Type: text/html
----- Blank line ------
<html>
<head><title>My Home</title></head>
<body><h1>This is my Home Page</h1>
</body></html>
```

HTTP Server



Протокол HTTP

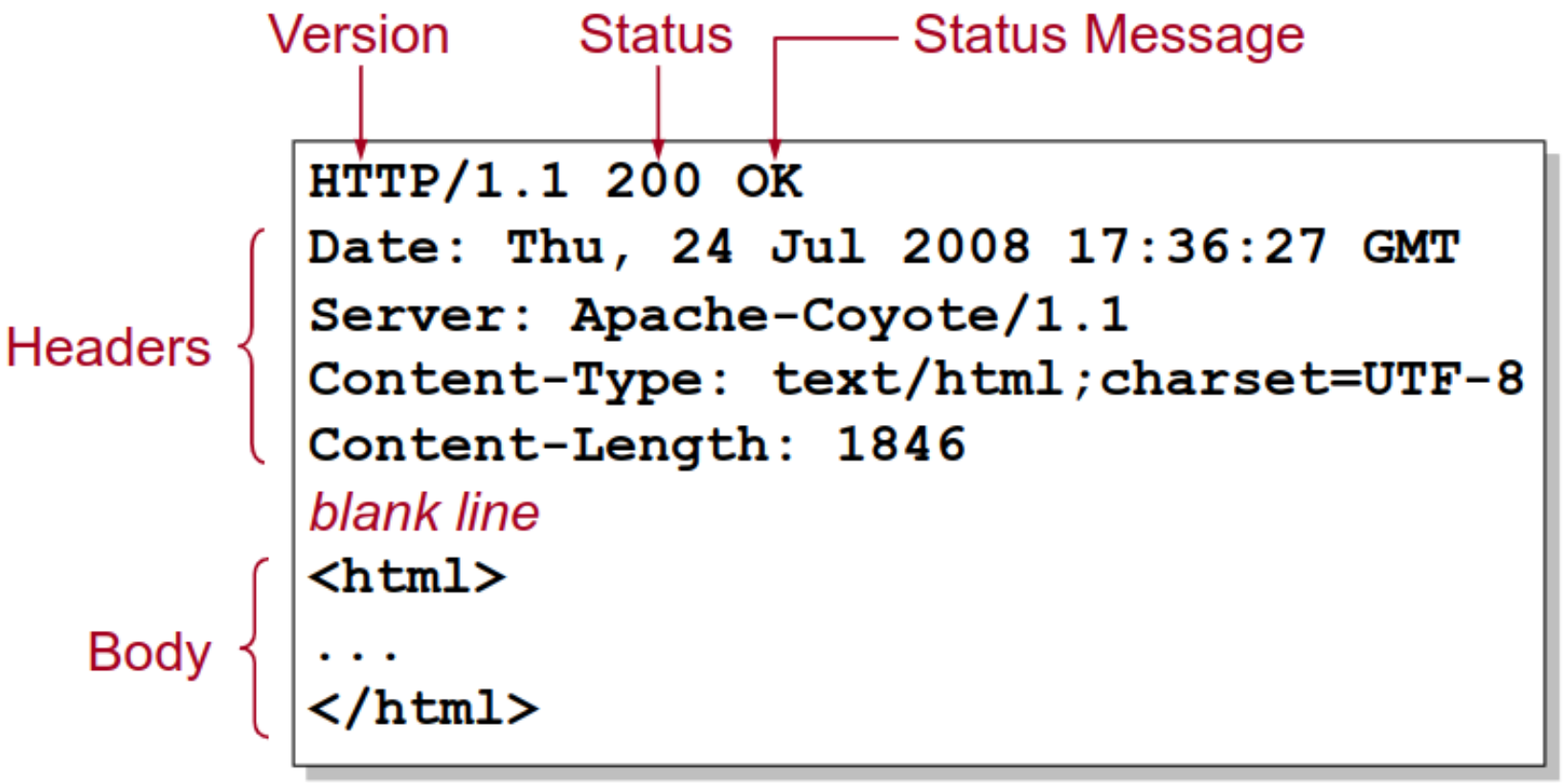
Методы

OPTIONS - Возвращает методы HTTP, которые поддерживаются сервером.
GET - Запрашивает содержимое указанного ресурса.
HEAD - Аналогичен методу GET, но в ответе отсутствует тело.
POST - Передаёт пользовательские данные заданному ресурсу.
PUT - Загружает указанный ресурс на сервер.
DELETE - Удаляет указанный ресурс.
TRACE - Возвращает полученный запрос так, что клиент может увидеть, что промежуточные сервера добавляют или изменяют в запросе.
CONNECT - Для использования вместе с прокси-серверами, которые могут динамически переключаться в туннельный режим SSL.



https://stackoverflow.com/questions/15772355/how-to-send-an-http-request-using-telnet

HTTP Response



Документация по HTTP

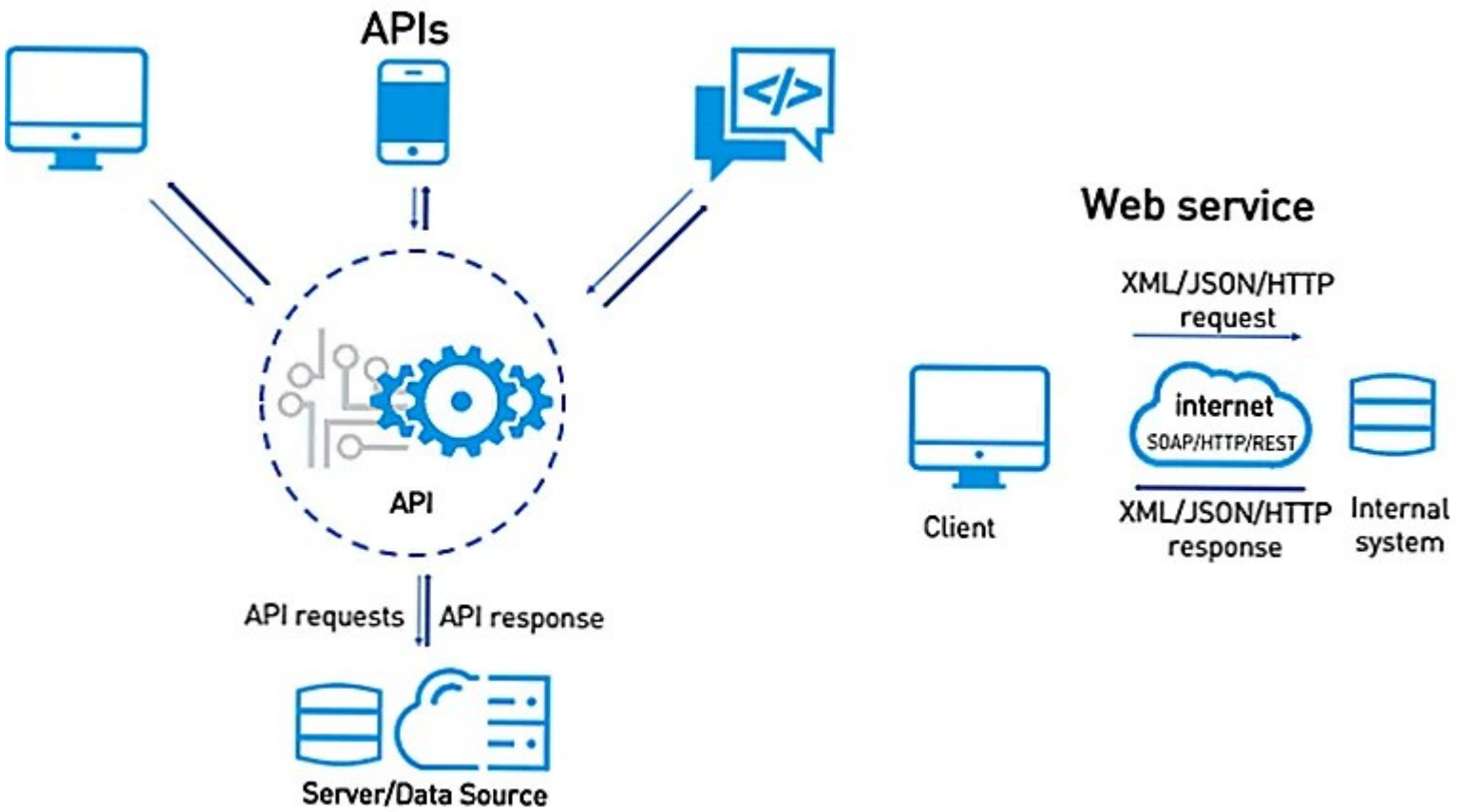
- https://developer.mozilla.org/ru/docs/Web/HTTP
- https://datatracker.ietf.org/doc/html/rfc2616

Web service

- Web service – сервис (набор методов) , предоставляемый приложением и доступный по сети
- Стандартизированный способ взаимодействия разнородных приложений
- Предаставление услуг для любого приложения

Web service

- Web service – сервис (набор методов) , предоставляемый приложением и доступный по сети
- Стандартизированный способ взаимодействия разнородных приложений
- Предаставление услуг для любого приложения


APIs
API
API requests
API response
Server/Data Source
Web service
XML/JSON/HTTP
request
internet
SOAP/HTTP/REST
XML/JSON/HTTP
response
Client
Internal
system

Что такое REST ?

- Технологию REST впервые представил один из авторов HTTP-протокола Рой Филдинг (Roy Fielding) в своей диссертации в Калифорнийском университете в Ирвайне (2000 год).
- REST (Representational state transfer) – это стиль архитектуры программного обеспечения для распределенных систем.
- REST - **НЕ** протокол
- REST является простым интерфейсом управления информацией
- Системы, поддерживающие REST, называются RESTful-системами

Принципы проектирования RESTful Web-сервисов

- Явное использование HTTP-методов.
- Взаимодействие между сервером и клиентом не хранит состояние (stateless).
- Предоставление URI, аналогичных структуре каталогов.
- Передача данных в XML, JavaScript Object Notation (JSON).

Какие методы HTTP задействует REST

SQL(CRUD)	REST(HTTP)
SELECT	**GET**
INSERT	**POST**
UPDATE	**PUT** **PATCH**
DELETE	**DELETE**

REST(URI) API design

URI	HTTP-метод	Действие
/student/list	GET	получить список всех студентов
/student/{id}	GET	получить студента по id
/student/{id}	POST	добавить студента (данные в теле запроса)
/student/	PUT PATCH	изменить студента (данные в теле запроса)
/student/{id}	DELETE	удалить студента

Протокол SOAP

- SOAP (Simple Object Access Protocol) - протокол обмена данными относится к подмножеству протоколов основанных на так называемой парадигме RPC (Remote Procedure Call, удалённый вызов процедур) антиподом которой является REST (Representational State Transfer, передача репрезентативного состояния)

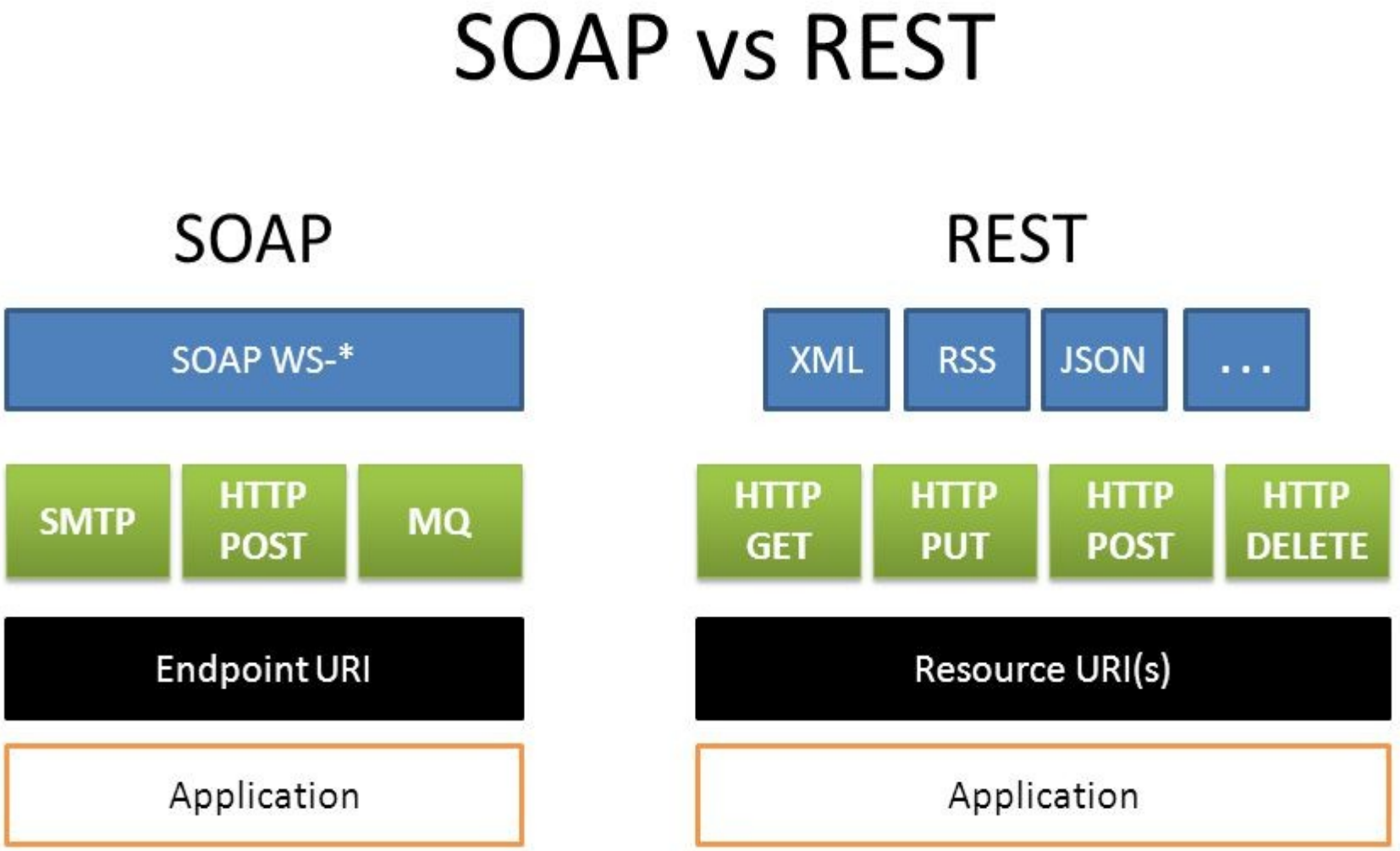
  Пример.

  https://yandex.ru/dev/direct/doc/dg-v4/concepts/SOAP.html

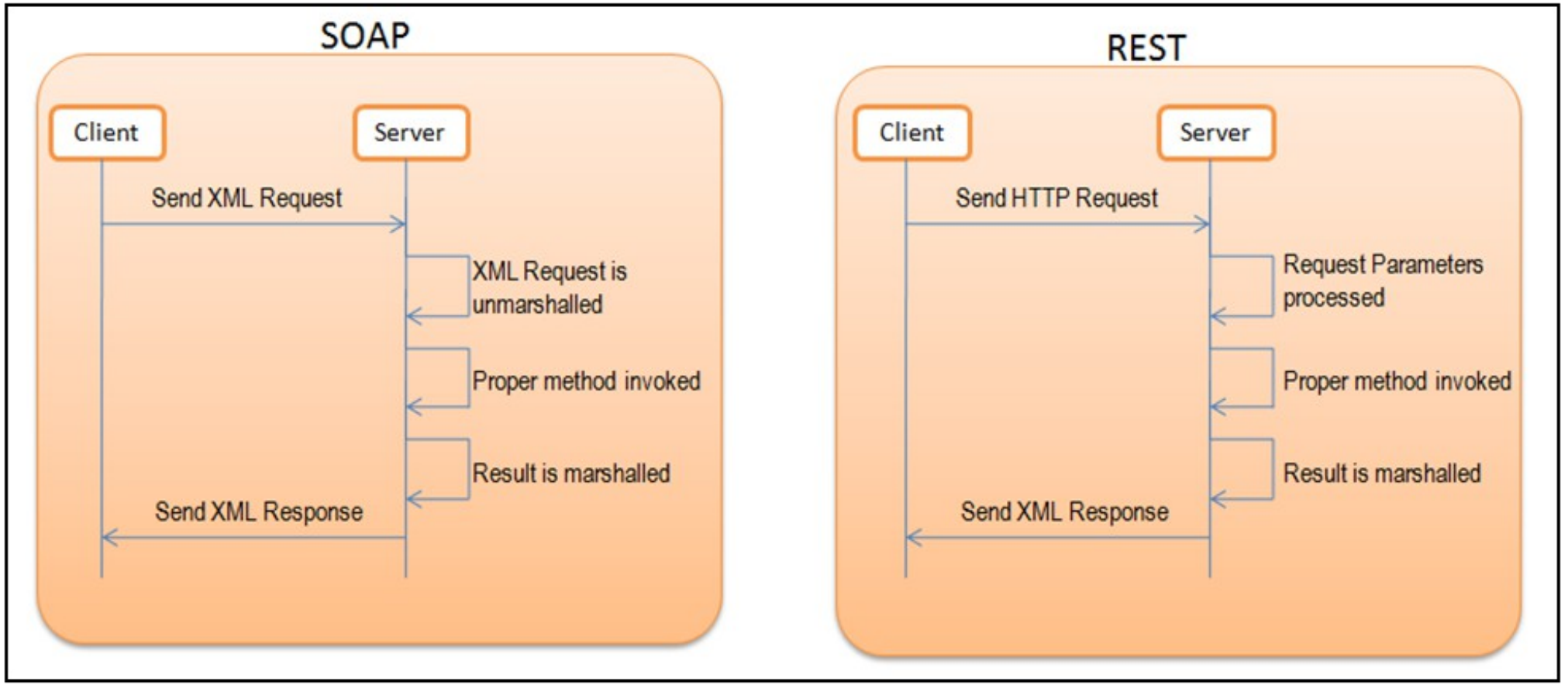
* https://www.w3.org/TR/wsdl.html

WSDL is an XML format for describing network services

Выбор между REST или SOAP



Взаимодействие



SOAP vs REST

Simple **O**bject **A**ccess **P**rotocol

REpresentational **S**tate **T**ransfer

- is a protocol
- uses services interfaces
- Works on top of any communication Protocol, even asynchronously (TCP HTTP FTP SMTP)
- Only XML
- Security and authorization are part of the protocol
- Fully described and provide information to users (WSDL)

is an architectural style

URL based services

HTTP/HTTPS only

JSON , XML, TEXT, HTML

Authorization and security must be implemented separately

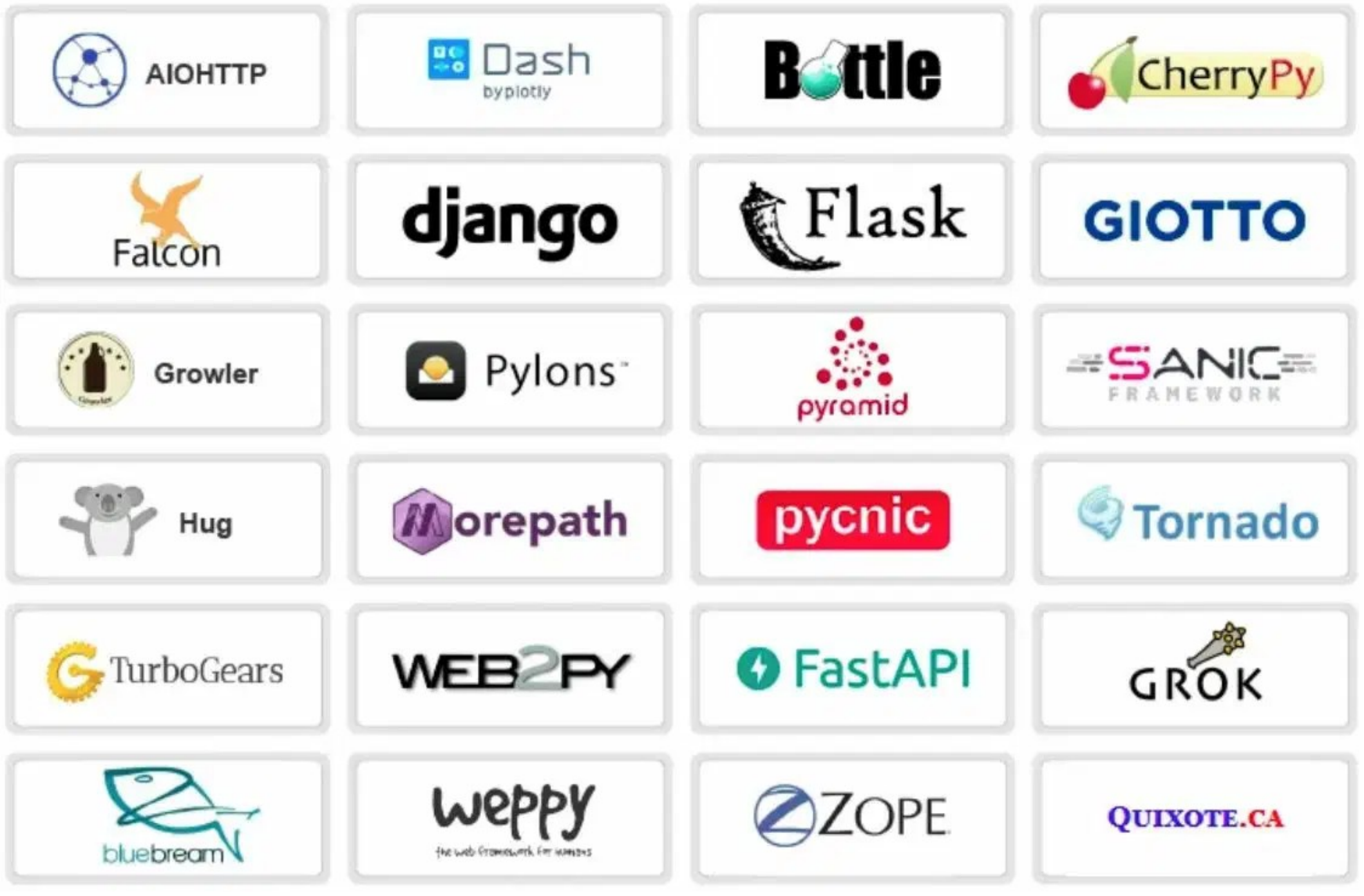
Easy to implement

Пример вызова сервера погоды по SOAP (XML) протоколу

```
import requests
url="http://wsf.cdyne.com/WeatherWS/Weather.asmx?WSDL"
#headers = {'content-type': 'application/soap+xml'}
headers = {'content-type': 'text/xml'}
body = """<?xml version="1.0" encoding="UTF-8"?>
         <SOAP-ENV:Envelope
xmlns:ns0="http://ws.cdyne.com/WeatherWS/"
xmlns:ns1="http://schemas.xmlsoap.org/soap/envelope/"
            xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
            <SOAP-ENV:Header/>
              <ns1:Body><ns0:GetWeatherInformation/></ns1:Body>
         </SOAP-ENV:Envelope>"""

response = requests.post(url,data=body,headers=headers)
print response.content
```

Выбор микрофреймворка (https://clck.ru/34SPWm)



REST on FLASK

https://flask.palletsprojects.com/en/2.1.x/#user-s-guide

```
# A Minimal Application
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

Запуск приложения

BASH

```
$ export FLASK_APP=hello
$ flask run
* Running on http://127.0.0.1:5000/
```

Маршруты (Routing)

```
from markupsafe import escape

@app.route('/user/<username>')
def show_user_profile(username):
    # show the user profile for that user
    return f'User {escape(username)}'

@app.route('/post/<int:post_id>')
def show_post(post_id):
    # show the post with the given id, the id is an
integer
    return f'Post {post_id}'

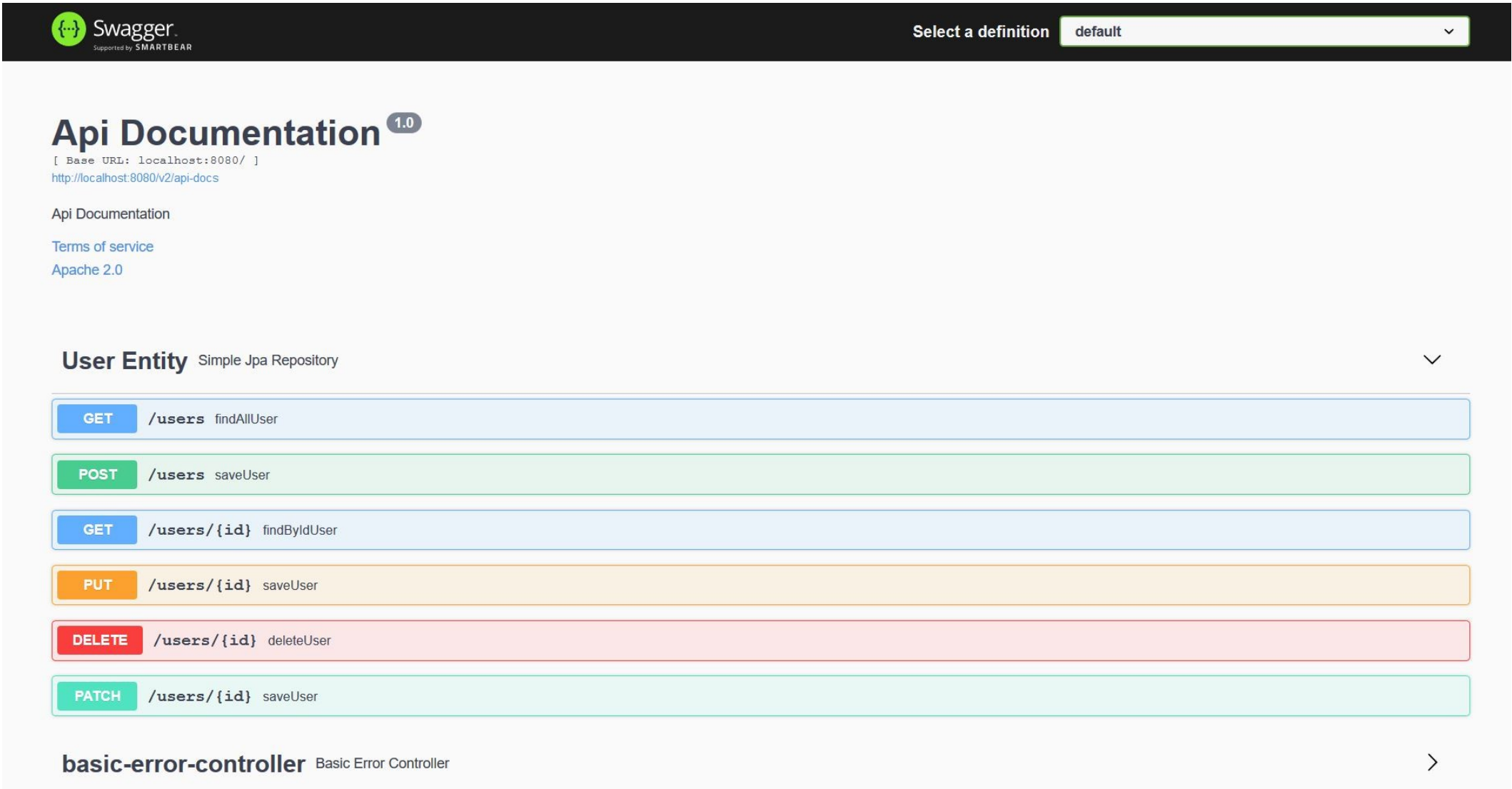
@app.route('/path/<path:subpath>')
def show_subpath(subpath):
    # show the subpath after /path/
    return f'Subpath {escape(subpath)}'
```

HTTP методы

```
from flask import request


@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return do_the_login()
    else:
        return show_the_login_form()
```

Swagger - REST документация



Базовая структура OPENAPI 3.0

```yaml
openapi: 3.0.0
info:
  title: Sample API
  description: Optional multiline or single-line description in [CommonMark](http://commonmark.org/help/) or HTML.
  version: 0.1.9

servers:
  - url: http://api.example.com/v1
    description: Optional server description, e.g. Main (production) server
  - url: http://staging-api.example.com
    description: Optional server description, e.g. Internal staging server for testing

paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in CommonMark or HTML.
      responses:
        '200':    # status code
          description: A JSON array of user names
          content:
            application/json:
              schema:
                type: array
                items:
                  type: string
```

Мне нужно описывать свой REST в YAML ?

Flask API Documentation using Flask-Restx

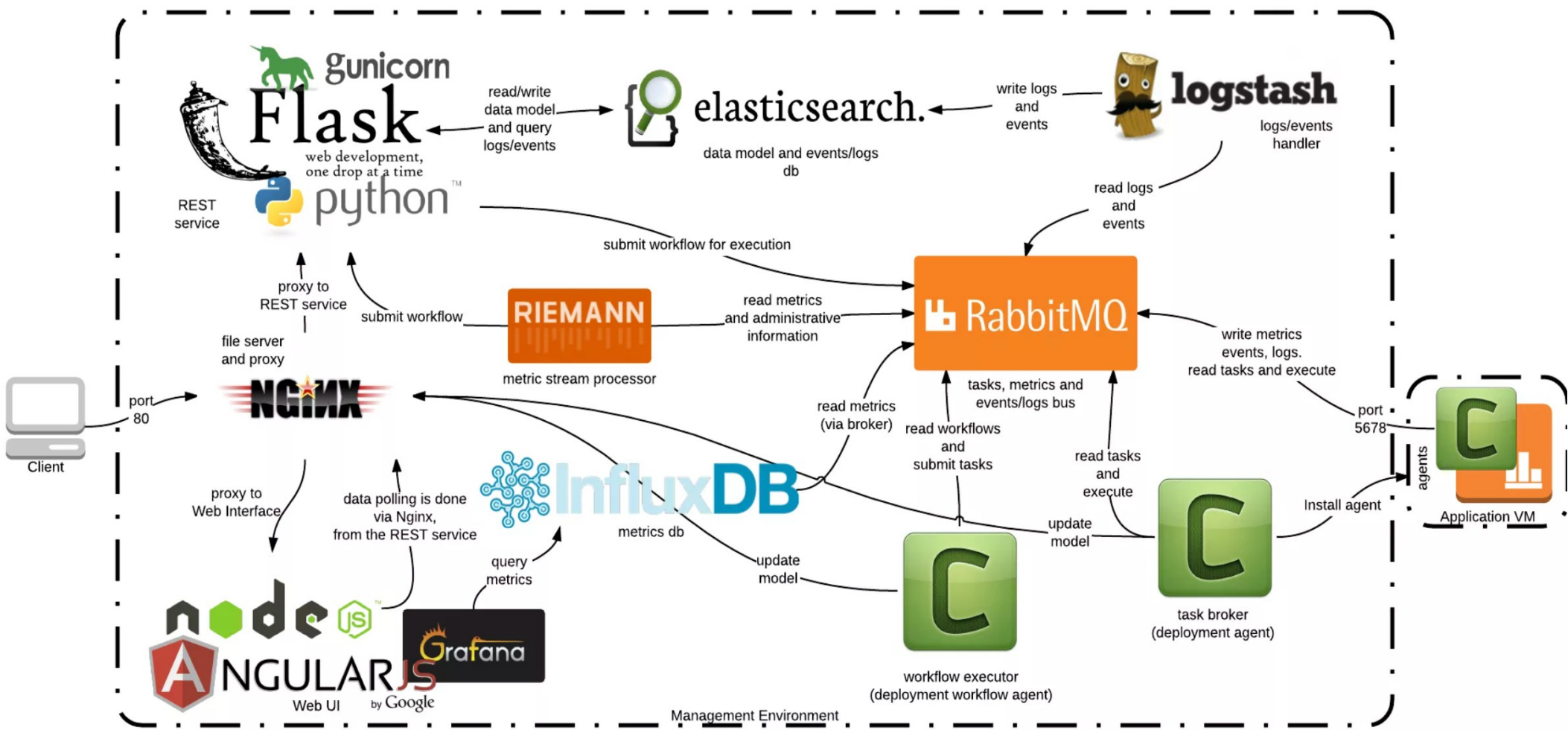
https://flask-restx.readthedocs.io/en/latest/quickstart.html

```
#При установки и добалении библиотеки restx
#документация на REST API будет генерироваться
#автоматически.

from flask import Flask, request
from flask_restx import Api, Resource, reqparse


app = Flask(__name__)
api = Api(app)
@api.route('/hello')
class HelloWorld(Resource):
    def get(self):
        return 'hello'
if __name__ == '__main__':
    app.run()
```

Что из себя представляет web нового поколения?


gunicorn
Flask
web development, one drop at a time
python
REST service
read/write data model and query logs/events
elasticsearch.
data model and events/logs db
write logs and events
logstash
logs/events handler
read logs and events
submit workflow for execution
RabbitMQ
proxy to REST service
submit workflow
RIEMANN
metric stream processor
read metrics and administrative information
file server and proxy
NGINX
port 80
Client
read metrics (via broker)
read workflows and submit tasks
tasks, metrics and events/logs bus
write metrics events, logs. read tasks and execute
port 5678
agents
Application VM
proxy to Web Interface
data polling is done via Nginx, from the REST service
InfluxDB
metrics db
query metrics
update model
update model
read tasks and execute
Install agent
task broker (deployment agent)
workflow executor (deployment workflow agent)
node
ANGULARJS
by Google
Web UI
Grafana
Management Environment

Заключение

Web-сервисы — это новая технология интеграции Web-приложений. На текущий момент она используется в корпоративных приложениях для предоставления интерфеса взаимодействия с бизнес приложением.